

1. Introduction to Dictionaries

In programming, we often need to store data in a meaningful way.

Consider student information:

- Name
- Roll Number
- Age
- Marks
- Course

Using lists becomes confusing because we must remember the position of each value.

Python provides **Dictionaries** to solve this problem.

A dictionary stores data in **key-value pairs**.

2. Why Dictionaries are Important

Dictionaries help us:

- Store data with meaningful labels
 - Access information quickly
 - Organize records efficiently
 - Represent real-world objects
 - Build databases and applications
-

Real-Life Example

Student Record:

Field	Value
-------	-------

Name	Rahul
------	-------

Roll No	101
---------	-----

Age	18
-----	----

Marks	85
-------	----

This structure is best represented using a dictionary.

3. What is a Dictionary?

A **Dictionary** is a collection of data stored as key-value pairs.

Syntax

```
dictionary_name = {  
    key1: value1,  
    key2: value2  
}
```

Example

```
student = {  
    "name": "Rahul",  
    "age": 18,  
    "marks": 85  
}
```

Output

```
{  
  'name': 'Rahul',  
  'age': 18,  
  'marks': 85  
}
```

4. Characteristics of Dictionaries

Key-Value Pair Structure

Every value has a key.

```
student = {  
    "name": "Rahul"  
}
```

Mutable

Dictionary values can be modified.

```
student["name"] = "Amit"
```

No Duplicate Keys

```
data = {  
    "name": "Rahul",  
    "name": "Amit"  
}
```

Output:

```
{'name': 'Amit'}
```

Latest value replaces the previous one.

Multiple Data Types Allowed

```
student = {  
    "name": "Rahul",  
    "age": 18,  
    "marks": 85.5,  
    "passed": True  
}
```

5. Creating Dictionaries

Empty Dictionary

```
student = {}
```

Dictionary with Data

```
student = {  
    "name": "Priya",  
    "roll_no": 102,  
    "marks": 90  
}
```

Using dict() Function

```
student = dict(  
    name="Rahul",  
    age=18  
)
```

6. Accessing Dictionary Values

Values are accessed using keys.

Example

```
student = {  
    "name": "Rahul",  
    "marks": 85  
}  
  
print(student["name"])
```

Output

Rahul

Using get() Method

```
print(student.get("marks"))
```

Output:

85

Advantage of get()

If key does not exist:

```
print(student.get("city"))
```

Output:

None

No error occurs.

7. Adding Elements to a Dictionary

New key-value pairs can be added easily.

Example

```
student = {  
    "name": "Rahul"  
}  
  
student["marks"] = 88  
  
print(student)
```

Output

```
{  
  'name': 'Rahul',  
  'marks': 88  
}
```

8. Updating Dictionary Values

Existing values can be modified.

Example

```
student = {  
    "name": "Rahul",  
    "marks": 80  
}  
  
student["marks"] = 95  
  
print(student)
```

Output

```
{  
  'name': 'Rahul',  
  'marks': 95  
}
```

9. Removing Elements from a Dictionary

pop()

Removes specific key.

```
student = {  
    "name": "Rahul",  
    "marks": 85  
}
```

```
student.pop("marks")
```

```
print(student)
```

Output

```
{'name': 'Rahul'}
```

del Keyword

```
del student["name"]
```

clear()

Removes all items.

```
student.clear()
```

Output:

```
{}
```

10. Dictionary Methods

Method Purpose

keys() Returns all keys

values() Returns all values

items() Returns key-value pairs

get() Retrieves value safely

pop() Removes item

clear() Removes all items

update() Updates dictionary

Example

```
student = {  
    "name": "Rahul",  
    "marks": 85  
}  
  
print(student.keys())  
print(student.values())  
print(student.items())
```

Output

```
dict_keys(['name', 'marks'])  
  
dict_values(['Rahul', 85])  
  
dict_items([  
('name', 'Rahul'),  
('marks', 85)  
])
```

11. Traversing Dictionaries

Using loops.

Loop Through Keys

```
student = {  
    "name": "Rahul",  
    "marks": 85  
}  
  
for key in student:  
    print(key)
```

Output

```
name  
marks
```

Loop Through Values

```
for value in student.values():  
    print(value)
```

Output

```
Rahul  
85
```

Loop Through Both

```
for key, value in student.items():  
    print(key, ":", value)
```

Output

```
name : Rahul  
marks : 85
```

12. Nested Dictionaries

A dictionary inside another dictionary is called a Nested Dictionary.

Example

```
students = {  
  
    "student1": {  
        "name": "Rahul",  
        "marks": 85  
    },  
  
    "student2": {  
        "name": "Priya",  
        "marks": 92  
    }  
}
```

Accessing Nested Data

```
print(students["student1"]["name"])
```

Output

Rahul

13. Student Records Using Dictionaries

One of the most common uses of dictionaries is storing student records.

Example

```
student = {  
    "roll_no": 101,  
    "name": "Rahul",  
    "age": 18,  
    "course": "Python",  
    "marks": 89  
}
```

Display Student Record

```
for key, value in student.items():  
    print(key, ":", value)
```

Output

```
roll_no : 101  
name : Rahul  
age : 18  
course : Python  
marks : 89
```

Student Result Program

```
student = {  
    "name": "Rahul",  
    "marks": 75  
}
```

```
if student["marks"] >= 35:
    print("Pass")
else:
    print("Fail")
```

Output

Pass

14. Common Dictionary Programs

Program 1: Count Total Keys

```
student = {
    "name": "Rahul",
    "age": 18,
    "marks": 85
}
```

```
print(len(student))
```

Output:

3

Program 2: Check Key Exists

```
student = {
    "name": "Rahul"
}
```

```
if "name" in student:
    print("Found")
```

Output:

Found

Program 3: Display All Values

```
student = {
    "name": "Rahul",
    "marks": 85
}
```

```
}  
  
for value in student.values():  
    print(value)
```

Program 4: Update Student Marks

```
student = {  
    "name": "Rahul",  
    "marks": 70  
}  
  
student["marks"] = 90  
  
print(student)
```

Output:

```
{  
    'name': 'Rahul',  
    'marks': 90  
}
```

15. Difference Between Lists, Tuples, Sets, and Dictionaries

Feature	List	Tuple	Set	Dictionary
Ordered	Yes	Yes	No	Yes
Mutable	Yes	No	Yes	Yes
Duplicates	Yes	Yes	No	Keys No
Indexing	Yes	Yes	No	Keys Used
Syntax	[]	()	{}	{key:value}

16. Common Errors in Dictionaries

Accessing Missing Key

```
student["city"]
```

Produces:

KeyError

Safe Alternative

```
student.get("city")
```

Forgetting Colon

Wrong:

```
student = {  
    "name" "Rahul"  
}
```

Correct:

```
student = {  
    "name": "Rahul"  
}
```

Duplicate Keys

```
student = {  
    "name": "Rahul",  
    "name": "Amit"  
}
```

Only latest value remains.

17. Advantages of Dictionaries

- Fast data retrieval
 - Easy record management
 - Real-world data representation
 - Flexible structure
 - Supports nested data
 - Useful in large applications
-

18. Real-Life Applications

Dictionaries are widely used in:

Student Management Systems

```
student = {  
    "name": "Rahul",  
    "marks": 90  
}
```

Banking Applications

```
account = {  
    "account_no": 12345,  
    "balance": 50000  
}
```

Employee Management

```
employee = {  
    "id": 101,  
    "name": "Amit"  
}
```

E-Commerce Websites

```
product = {  
    "name": "Laptop",  
    "price": 55000  
}
```

APIs and JSON Data

Most web applications use dictionaries to exchange data.

19. Conclusion

Dictionaries are one of the most powerful and widely used data structures in Python. They store information using **key-value pairs**, making data easy to understand and manage.

Student records are a perfect real-world example of dictionaries because each piece of information can be stored with a meaningful key such as name, roll number, marks, or course.

Mastering dictionaries is essential because they are heavily used in web development, databases, APIs, data science, Artificial Intelligence, automation, and software development. They form the foundation for managing structured data in real-world applications.

